

SMARTBAND V001

Hardware Integration & System Demonstration Report

Task 3 — Peripheral Integration Phase · Raspberry Pi 5

Student: Masrur

Institution: Hochschule Hamm-Lippstadt (HSHL)

Repository: Band01_Rev1 (Private GitHub)

Date: June 2026

Objective: Establish complete end-to-end health monitoring pipeline — BT, OLED, LED, buzzer, live plot, ThingsBoard cloud

HARDWARE COMPONENTS

Component	Type	Interface	GPIO / Address
Raspberry Pi 5	Host Platform	—	—
Pmod BT2 (RN-42)	Bluetooth Module	UART	GPIO14 (TX), GPIO15 (RX)
SSD1306 128×64	OLED Display	I2C	0x3C · SDA: GPIO2, SCL: GPIO3
RGB LED	Common Cathode	GPIO	Red: GPIO27 · Green: GPIO26
Active Buzzer	Alarm Module	GPIO	GPIO17
T-Shaped Extension Board	GPIO Extender	Ribbon Cable	40-pin breakout

WIRING & PIN MAPPING

OLED SSD1306 (I2C)

VCC	3V3 (left)	e5
GND	GND (left)	e9
SDA	SDA1 (left)	e6
SCL	SCL1 (left)	e7

Pmod BT2 Bluetooth (UART)

Pin 2 (RX)	TXD0 (right)	f8
Pin 3 (TX)	RXD0 (right)	f9
Pin 5 (GND)	GND (right)	i21
Pin 6 (VCC)	3V3 (left)	e13

RGB LED & Buzzer (GPIO)

Component	GPIO	Ext. Label	Breadboard
RGB LED Red	GPIO27	GPIO27	a40 via 330Ω
RGB LED Green	GPIO26	GPIO26	a42 via 330Ω
RGB LED GND	GND	GND	a41
Buzzer (+)	GPIO17	GPIO17	a30
Buzzer (-)	GND	GND	a34

SOFTWARE ARCHITECTURE & DATA PIPELINE

System Files:

`smartband_full_v3.py`

Main system — sensor simulation, OLED, LED, buzzer, BT, ThingsBoard, CSV

`smartband_plot.py`

Real-time matplotlib dashboard — reads `sensor_data.csv` and plots live graphs

`font5x8.bin`

Bitmap font required by `adafruit_ssd1306` for OLED text rendering

Data Flow (every 2 seconds):

Sensor Simulator
HR · Temp · Accel

`sensor_data.csv`
→ `live_plot.py`

ThingsBoard MQTT
`mqtt.eu.thingsboard`

Pmod BT2 UART
→ Android SPP

OLED SSD1306
HR / T / A display

RGB LED + Buzzer
Emergency alert

Emergency Logic

- Heart Rate > 100 BPM → Red LED flash + Buzzer ON + OLED emergency screen
- Temperature > 38.5°C → Red LED flash + Buzzer ON + OLED emergency screen
- Acceleration > 2.0 G → Red LED flash + Buzzer ON + OLED emergency screen

BLUETOOTH INTEGRATION — Pmod BT2 (RN-42)

Module Details:

Chip	Microchip RN-42
Device Name	RNBT-0793
MAC Address	00:06:66:F9:07:93
Baud Rate	115200 bps, 8N1
Profile	Classic Bluetooth SPP
Mode	Slave (waits for incoming)
Auth	Disabled (SA,0) — open pairing
UART Device	/dev/serial0

✓ Android verification: Paired with RNBT-0793 via Serial Bluetooth Terminal — live data stream confirmed

Connection Method (v3)

Pi UART (/dev/serial0) → Pmod BT2 → Wireless SPP to Android.
Eliminates rfcomm dependency. Any Android device can connect directly to RNBT-0793 and receive live sensor data via Serial Bluetooth Terminal app.

Android Data Format (Verified ✓)

```
HR:75 T:36.6 A:1.2
```

```
!! EMERGENCY !! HR:115 T:39.2 A:2.8
```

OLED DISPLAY & THINGSBOARD CLOUD

SSD1306 Display Screens:

Sensor View HR, Temp, Accel, Timestamp	Normal — alternates every 2s
Status View Date, Time, ThingsBoard status	Normal — alternates every 2s
Emergency View !! EMERGENCY !!, all sensor values	Any threshold exceeded
Startup Screen Smartband V001, HSHL Project, Starting...	System boot
Shutdown Screen System OFF, Goodbye!, HSHL Project	Keyboard interrupt

ThingsBoard Cloud (MQTT)

Host:	mqtt.eu.thingsboard.cloud
Port:	1883
Protocol:	MQTT (paho-mqtt v2)
Topic:	v1/devices/me/telemetry
Fields:	heartRate, temperature, acceleration, status, hr_limit, temp_limit, acc_limit
Offline:	Graceful — all local ops continue (OLED, LED, buzzer, CSV, BT)

TESTING & VERIFICATION — 14 Tests All Passed

OLED Detection i2cdetect -y 1	0x3C detected	✓	Buzzer pinctrl GPIO17 toggle	Audible buzz confirmed	✓
OLED Display adafruit_ssd1306 text render	Text displayed correctly	✓	LED Red pinctrl GPIO27 toggle	Red illuminated	✓
Pmod BT2 Power Visual — LED blinking	Slow blink = ready	✓	LED Green pinctrl GPIO26 toggle	Green illuminated	✓
Pmod BT2 UART Python serial + \$\$\$ command	CMD response received	✓	Emergency Mode Simulated spike (step % 8)	LED flash + buzzer + OLED	✓
UART Loopback TXD0 → RXD0 jumper	b'hello' echoed back	✓	ThingsBoard MQTT paho-mqtt publish	Data visible on dashboard	✓
BT Pairing (Pi) bluetoothctl pair	Paired and trusted	✓	Android BT Serial Bluetooth Terminal app	Live data stream received	✓
BT Data Transfer rfcomm + Python serial	Data received on UART	✓	Live Graph smartband_plot.py	All 3 charts updating live	✓

ISSUES ENCOUNTERED & RESOLUTIONS

Issue	Root Cause	Resolution
OLED not on i2cdetect	I2C not enabled on Pi	raspi-config → Interface Options → I2C
OLED blank screen	Wires in column j (not on extender)	Moved all wires to column e
Pmod no response to \$\$\$	Pi 5 UART not on GPIO14/15 by default	Added enable_uart=1 to /boot/firmware/config.txt
BT CMD mode no response	Wrong serial device path	Used /dev/serial0 after enable_uart=1 reboot
BT pairing failed (passkey)	RN-42 uses fixed PIN, not passkey	Set SA,0 (no auth) + SP,1234 (PIN) via UART
Phone could not connect	Pi rfcomm occupied the module	Cleared SR,Z, switched to direct UART in v3
LED Green not lighting	Signal wire in GND row (a41)	Moved wire to LED Green row (a42)
font5x8.bin missing	File not in working directory	Downloaded from Adafruit GitHub to ~/smartband/

CONCLUSION & NEXT STEPS

Capabilities Delivered:

- ✓ Bluetooth SPP via Pmod BT2 — Android verified
- ✓ SSD1306 OLED — 5 display screens
- ✓ RGB LED + Buzzer emergency alerts
- ✓ ThingsBoard MQTT cloud telemetry
- ✓ Real-time matplotlib dashboard (3 charts)
- ✓ 14 tests — all passed ✓

Next Steps:

- Assemble Band01_Rev1 Smartband PCB hardware
- Activate sensor calls in `simple_peripheral.c` and `i2c_sensors.c`
- Replace simulated data with real BLE data from Smartband
- Implement BLE scanning on Pi (bluepy/bleak) for CHAR1 0xFFFF1
- Integrate OledThread & AppThread (currently disabled in `main.c`)
- Replace 0x7788 placeholder with real sensor readings